

DSBA 6190 • Cloud Computing for Data Analysis

AWS Lab 2

Distributed Computing for Data Processing

Docker • Amazon ECR • Amazon EKS • Amazon EMR (Spark) • Amazon Athena • AWS Glue Data Catalog

Begin Week 5 (Feb 9) • Due Week 7 (Feb 23) • 75 Points

Spring 2026

UNC Charlotte • The Dubois Center (Uptown)

Dr. Frazier Smith

This lab is designed for use within the AWS Academy Learner Lab environment. Service availability, instance types, and IAM constraints reflect the Learner Lab sandbox. Review the constraints table on the next page before starting.

Lab 2: Distributed Computing for Data Processing

Services: Docker, Amazon ECR, Amazon EKS, Amazon EMR (Spark), Amazon Athena, AWS Glue Data Catalog

Schedule: Begin Week 5 (Feb 9) • Due Week 7 (Feb 23)

Points: 75

Learner Lab Constraints

Review these restrictions before starting. Exceeding instance limits can result in immediate account deactivation and loss of all work.

Service	Constraint
EC2 / EKS / EMR	Instance types limited to nano, micro, small, medium, and large. Maximum 9 concurrent EC2 instances across all services; maximum 32 vCPU total. Exceeding 20 running instances deactivates the account immediately.
EKS	Must use LabEksClusterRole (not LabRole) for both the cluster role and the node group role.
EMR	Use m4.large instances. Clusters do not survive session ends—all EC2 instances are stopped when the timer hits 0:00 and EMR does not support stop/restart. Write results to S3 before ending your session.
ECR	LabRole has read-only access; your console user has write access. Push images from the CLI using your console credentials.
Glue	Uses LabRole. Worker type limited to G.1X or Standard; maximum 10 workers.
Athena	Uses LabRole. You must configure an S3 location for query results before running queries.

Session Management

This lab spans multiple Learner Lab sessions. Each part produces output stored in S3, which persists across sessions. Compute resources (EC2 instances, EKS clusters, EMR clusters) do not survive session ends and must be cleaned up before you stop. The following table summarizes what persists and what does not:

Resource	Survives Session End?	Action Required
S3 buckets and data	Yes	No action needed
ECR images	Yes	No action needed
Glue databases/tables	Yes	No action needed
Athena settings	Yes	No action needed
EC2 instances	Stopped (not terminated)	Can restart in next session
EKS clusters	No	Delete before stopping
EMR clusters	No	Terminate before stopping
CloudShell binaries	Home dir yes; /usr/local/bin no	Reinstall eksctl if needed

Each part includes a stopping-point box at the end that tells you exactly what to clean up before ending your session. Each part (except Part 1) also includes a resume box at the top with instructions for starting from a new session.

Lab Overview

This lab builds an end-to-end distributed data processing pipeline for a realistic analytics scenario. You will work with a public trip-record dataset (such as NYC Taxi or Chicago rideshare data) that is too large to process efficiently on a single machine. The pipeline moves through four stages: containerizing a data-cleaning script so it can run repeatably in any environment, orchestrating that container on Kubernetes for scalable batch processing, running heavier analytical transformations with Spark on EMR, and making the final results queryable through Athena so analysts can run SQL against the processed data without managing any infrastructure.

Each part produces output that feeds into the next, mirroring how data engineering teams build layered data platforms in practice: raw ingestion, cleaning, transformation, and an analytics-ready query layer.

Part 1: Containerizing a Data Cleaning Pipeline

Estimated time: 45–60 minutes

Objectives

- Write a Dockerfile for a Python data-cleaning script that handles common quality issues in the raw dataset (null values, outliers, schema normalization)
- Build and test the container image locally, verifying that the cleaned output conforms to an expected schema
- Push the image to Amazon ECR so it can be pulled by orchestration services in subsequent parts

Tasks

1. Create an S3 bucket to hold your lab data. In the AWS Console, navigate to S3 > Create bucket. Name it something unique such as dsba6190-yourname-lab2. Keep the default settings (us-east-1, ACLs disabled, Block all public access enabled) and choose Create bucket.
2. Upload the provided cleaning script (clean_trips.py), sample data partition (sample_trips.csv), and the three monthly data files (trips_2023_01.csv, trips_2023_02.csv, trips_2023_03.csv) to your bucket. Upload the monthly files into partitioned prefixes for later use in Part 2:

```
# You can upload via the S3 console (drag and drop) or from CloudShell:

# Script and sample data (for Part 1 testing)
aws s3 cp clean_trips.py s3://dsba6190-yourname-lab2/
aws s3 cp sample_trips.csv s3://dsba6190-yourname-lab2/

# Monthly partitions (for Part 2 – upload now while you have the files)
aws s3 cp trips_2023_01.csv s3://dsba6190-yourname-lab2/raw/month=01/
aws s3 cp trips_2023_02.csv s3://dsba6190-yourname-lab2/raw/month=02/
aws s3 cp trips_2023_03.csv s3://dsba6190-yourname-lab2/raw/month=03/
```

Also upload the PySpark script (spark_aggregations.py) and zone lookup table (zone_lookup.csv) for Part 3. Uploading all files to S3 now means you will not need local copies in later sessions:

```
aws s3 cp spark_aggregations.py s3://dsba6190-yourname-lab2/scripts/
aws s3 cp zone_lookup.csv s3://dsba6190-yourname-lab2/reference/
```

3. Launch an EC2 instance for Docker work. Navigate to EC2 > Launch instances and configure as follows:

```
Name:                lab2-docker-build
AMI:                 Amazon Linux 2023 (Quick Start, first option)
Instance type:       t3.small
Key pair:             vockey
Network:             Default VPC, Auto-assign public IP = Enable
Security group:      Create new, allow SSH (port 22) from My IP
```

```
Under Advanced details:
IAM instance profile: LabInstanceProfile
```

Choose Launch instance. Wait for the instance state to show "Running" and the status check to show "2/2 checks passed" (typically 1–2 minutes).

4. Connect to the instance. Select the instance in the EC2 console, choose Connect, then use the EC2 Instance Connect tab (browser-based SSH). Alternatively, from CloudShell:

```
ssh -i ~/.ssh/labsuser.pem ec2-user@<public-ip>
```

5. Verify Docker is available and download your files from S3:

```
docker --version
# Should output Docker version 2x.x.x or similar

mkdir ~/lab2 && cd ~/lab2
aws s3 cp s3://dsba6190-yourname-lab2/clean_trips.py .
aws s3 cp s3://dsba6190-yourname-lab2/sample_trips.csv .
```

6. Create a Dockerfile in the ~/lab2 directory. The cleaning script uses pandas and pyarrow to read CSV, filter bad rows, and write Parquet. The container should accept S3 input/output paths as command-line arguments:

```
FROM python:3.11-slim

WORKDIR /app
RUN pip install --no-cache-dir pandas pyarrow boto3
COPY clean_trips.py .

ENTRYPOINT ["python", "clean_trips.py"]
```

7. Build and test the image locally against the sample data:

```
docker build -t trip-cleaner:latest .

# Test with local files mounted (not S3) for quick validation
docker run --rm \
  -v ~/lab2:/data \
  trip-cleaner:latest \
  --input /data/sample_trips.csv \
  --output /data/cleaned_sample.parquet
```

The script should print a summary: input row count, rows dropped by each filter (nulls, duration outliers, distance outliers), and final output row count. Verify the output Parquet file was created.

8. Create an ECR repository and push the image. From the EC2 instance terminal:

```
# Get your AWS account ID
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
REGION=us-east-1

# Create the repository
aws ecr create-repository --repository-name trip-cleaner --region $REGION

# Authenticate Docker to ECR
aws ecr get-login-password --region $REGION | \
  docker login --username AWS --password-stdin \
    $ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com

# Tag and push
```

```
docker tag trip-cleaner:latest \
    $ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/trip-cleaner:latest

docker push \
    $ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/trip-cleaner:latest
```

Wait for the push to complete. Verify the image appears in the ECR console: navigate to Amazon ECR > Repositories > trip-cleaner. You should see the "latest" tag with the image size and push date.

Deliverables

- Dockerfile and cleaning script (clean_trips.py)
- Screenshot of the docker run output showing input row count, rows filtered, and final row count
- Screenshot of the image listed in the ECR repository console

Stopping after Part 1

- Stop or terminate the EC2 instance: EC2 > Instances > select lab2-docker-build > Instance state > Stop instance.
- Your ECR image persists across sessions — you do not need to rebuild or re-push it.
- Your S3 bucket and all uploaded data (including the monthly partitions and Spark files) persist across sessions.
- You can safely end your Learner Lab session.

Part 2: Orchestrating Data Processing on Amazon EKS

Estimated time: 60–90 minutes (includes ~15 min cluster provisioning)

▶ Starting Part 2 in a new session?

- Start your Learner Lab session and open the AWS Console.
- Verify your S3 data is intact: navigate to S3, open your bucket, and confirm the raw/month=01/, raw/month=02/, and raw/month=03/ prefixes contain the monthly CSV files. (You uploaded these in Part 1, Step 2.)
- Verify your ECR image is intact: navigate to ECR > Repositories > trip-cleaner. The "latest" tag should still be listed.
- You do not need the EC2 instance from Part 1. If it is still in "Stopped" state, leave it stopped.
- Open CloudShell for the steps below.

Objectives

- Deploy the containerized cleaning pipeline as a Kubernetes Job on EKS, processing multiple data partitions in parallel
- Understand how container orchestration enables horizontal scaling of data processing workloads
- Verify the cleaned dataset in S3 is ready for downstream Spark processing

Tasks

1. Open CloudShell (the terminal icon at the top of the AWS Console). Install eksctl. CloudShell preserves your home directory between sessions, but /usr/local/bin is reset each time. Install eksctl to your home directory so it persists:

```
# Download eksctl
curl -sLO "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Linux_amd64.tar.gz"
tar -xzf eksctl_Linux_amd64.tar.gz

# Install to home directory (persists across CloudShell sessions)
mkdir -p ~/bin
mv eksctl ~/bin/
export PATH=$HOME/bin:$PATH

# Add to .bashrc so it's available in future sessions
echo 'export PATH=$HOME/bin:$PATH' >> ~/.bashrc

eksctl version
```

2. Create the EKS cluster with a managed node group:

```
eksctl create cluster \
  --name lab2-cluster \
  --region us-east-1 \
  --nodegroup-name data-nodes \
  --node-type t3.medium \
  --nodes 2 \
```

```
--managed \  
--with-oidc
```

This takes approximately 15 minutes. The eksctl tool provisions the cluster, VPC networking, and node group. When complete, it automatically configures kubectl.

Two t3.medium nodes use 4 vCPU and count as 2 of your 9 allowed instances. If you have the Docker-build EC2 instance from Part 1 still in "Running" state, stop it first (EC2 > Instances > select > Instance state > Stop instance).

3. Verify the cluster is accessible:

```
kubectl get nodes  
# Should show 2 nodes with STATUS = Ready
```

4. If eksctl did not assign the correct IAM roles, you may need to update the aws-auth ConfigMap to allow the node role. Check the EKS console under your cluster > Compute > Node groups to verify nodes registered. If nodes show "Not Ready," consult the instructor.
5. Create a Kubernetes Job manifest file (cleaning-job.yaml). This job runs three pods in parallel, each processing a different monthly partition. Each pod pulls the container image from ECR:

```
apiVersion: batch/v1  
kind: Job  
metadata:  
  name: trip-cleaning-batch  
spec:  
  parallelism: 3  
  completions: 3  
  completionMode: Indexed  
  template:  
    spec:  
      restartPolicy: Never  
      containers:  
      - name: cleaner  
        image: <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/trip-cleaner:latest  
        command:  
        - python  
        - clean_trips.py  
        - --input  
        - s3://dsba6190-yourname-lab2/raw/month=$(JOB_COMPLETION_INDEX_PLUS_1) /  
        - --output  
        - s3://dsba6190-yourname-lab2/cleaned/month=$(JOB_COMPLETION_INDEX_PLUS_1) /  
        env:  
        - name: AWS_DEFAULT_REGION  
          value: "us-east-1"
```

Replace <ACCOUNT_ID> with your actual AWS account ID. The JOB_COMPLETION_INDEX is 0-indexed; your cleaning script should map index 0 to month=01, index 1 to month=02, etc. You may handle this mapping either in the script itself or with an init container.

6. Apply the manifest and monitor the job:

```
kubectl apply -f cleaning-job.yaml
```



```
# Watch progress (updates every 5 seconds)
kubectl get jobs --watch

# Once pods appear, check logs for a specific pod
kubectl get pods
kubectl logs trip-cleaning-batch-0-xxxxxx
```

7. Verify the output. Each pod should have written a cleaned Parquet file to its respective S3 prefix:

```
aws s3 ls s3://dsba6190-yourname-lab2/cleaned/ --recursive
# Should show 3 Parquet files, one per month partition
```

8. Delete the cluster to preserve your budget. This step is required whether or not you plan to continue to Part 3 in this session:

```
eksctl delete cluster --name lab2-cluster --region us-east-1
# This takes ~5 minutes and removes the cluster, node group, and VPC resources
```

Do not skip this step. An idle EKS cluster with 2 t3.medium nodes costs approximately \$3.50/hour and counts against your instance limits. The EKS cluster does not survive session ends — if the session timer expires, the underlying EC2 nodes are stopped but the orphaned cluster control plane may cause issues when you return.

Deliverables

- Kubernetes Job manifest (cleaning-job.yaml)
- Screenshot of kubectl get jobs showing all completions succeeded
- Screenshot of aws s3 ls showing cleaned Parquet files for each month partition

Stopping after Part 2

- Delete the EKS cluster (Step 8 above). Confirm deletion is complete before ending your session.
- If the EC2 instance from Part 1 is still running, stop it.
- Your cleaned Parquet files in s3://dsba6190-yourname-lab2/cleaned/ persist across sessions. Part 3 reads from this location.
- The PySpark script and zone lookup file you uploaded to S3 in Part 1, Step 2 are already in place for Part 3.
- You can safely end your Learner Lab session.

Part 3: Analytical Transformations with Amazon EMR (Spark)

Estimated time: 60–75 minutes (includes ~10 min cluster startup)

▶ Starting Part 3 in a new session?

- Start your Learner Lab session and open the AWS Console.
- Verify your cleaned data from Part 2 is in S3: run `aws s3 ls s3://dsba6190-yourname-lab2/cleaned/ --recursive` in CloudShell. You should see 3 Parquet files.
- Verify your PySpark script and zone lookup are in S3: run `aws s3 ls s3://dsba6190-yourname-lab2/scripts/` and `aws s3 ls s3://dsba6190-yourname-lab2/reference/`. (You uploaded these in Part 1, Step 2.)
- Confirm no leftover EC2 instances or EKS clusters are running: check EC2 > Instances and EKS > Clusters. Stop or delete any orphaned resources before proceeding.
- You are ready to create the EMR cluster below.

Objectives

- Launch an EMR cluster and run a PySpark job that reads the cleaned data from Parts 1–2 and produces analytical aggregations
- Use Spark SQL to compute metrics that would be impractical to calculate row-by-row: trip volume by zone and hour, rolling averages, and percentile distributions
- Write the results as partitioned Parquet optimized for downstream Athena queries

Tasks

1. Create an EMR cluster. Navigate to EMR > Create cluster in the AWS Console and configure:

```
Cluster name:          lab2-spark-cluster
EMR release:           emr-7.x.x (latest available)
Application bundle:    Spark (Interactive or Custom – ensure Spark is selected)

Cluster configuration: Instance groups
  Primary:    m4.large (1 instance)
  Core:       m4.large (1 instance)
  Task:       m4.large (1 instance)

Cluster logs:  UNCHECK "Publish cluster-specific logs to Amazon S3"
               (This avoids S3 bucket creation permission errors)

EC2 key pair:  vockey

IAM roles:
  Amazon EMR service role:    EMR_DefaultRole
  EC2 instance profile for EMR: EMR_EC2_DefaultRole
```

Choose Create cluster. The cluster takes approximately 8–12 minutes to reach "Waiting" state. Monitor progress on the cluster detail page.

These 3 EMR instances count toward your 9-instance limit. Verify no other EC2 instances are running (EC2 > Instances). If the Docker-build instance from Part 1 is still in "Running" state, stop it first.

2. Add a Spark step to the cluster. Once the cluster shows "Waiting," choose the Steps tab, then Add step:

```
Step type:      Spark application
Name:           trip-aggregations
Deploy mode:    Cluster
Application location: s3://dsba6190-yourname-lab2/scripts/spark_aggregations.py
Arguments:
  --input s3://dsba6190-yourname-lab2/cleaned/
  --zones s3://dsba6190-yourname-lab2/reference/zone_lookup.csv
  --output s3://dsba6190-yourname-lab2/aggregated/

Action on failure: Continue
```

Choose Add step. The step will show "Pending" then "Running." Monitor it until it shows "Completed" (typically 5–10 minutes depending on data volume).

3. If the step fails, check the stderr log. In the Steps tab, choose the step name, then View logs > stderr. Common issues include S3 path typos and Python import errors. Fix the script, re-upload to S3, and resubmit the step.
4. Verify the output. Check that both output tables exist with the expected partition structure:

```
# Hourly aggregation (not partitioned – single directory)
aws s3 ls s3://dsba6190-yourname-lab2/aggregated/hourly/

# Daily summary (partitioned by date)
aws s3 ls s3://dsba6190-yourname-lab2/aggregated/daily_summary/
# Should show prefixes like date=2023-01-01/, date=2023-01-02/, etc.
```

5. Terminate the EMR cluster. In the EMR console, select the cluster and choose Terminate. Wait for the cluster status to change to "Terminated" before ending your session:

EMR clusters do not survive session ends. If the session timer expires with the cluster running, the EC2 instances will be force-stopped, which corrupts the cluster and may leave orphaned resources. Always terminate EMR clusters explicitly and verify your Spark output is in S3 before ending your session.

Deliverables

- PySpark script (spark_aggregations.py)
- Screenshot of the EMR step showing "Completed" status
- Screenshot of the S3 output directory showing the hourly and daily_summary tables with partitioned Parquet files

⏸ Stopping after Part 3

- Terminate the EMR cluster (Step 5 above). Wait for "Terminated" status in the EMR console.

- Verify no other EC2 instances are running.
- Your aggregated output in s3://dsba6190-yourname-lab2/aggregated/ persists across sessions. Part 4 reads from this location.
- You can safely end your Learner Lab session.

Part 4: Building an Analytics Query Layer with Athena

Estimated time: 30–45 minutes

Starting Part 4 in a new session?

- Start your Learner Lab session and open the AWS Console.
- Verify your Spark output from Part 3 is in S3: run `aws s3 ls s3://dsba6190-yourname-lab2/aggregated/ --recursive` in CloudShell. You should see Parquet files under both `hourly/` and `daily_summary/` prefixes.
- No compute resources need to be running for Part 4. Athena and Glue are serverless—you only need the S3 data.
- Navigate to Athena in the console to begin.

Objectives

- Create Glue Data Catalog tables over the Spark output so analysts can discover and query the data using standard SQL
- Run analytical queries in Athena that answer specific business questions about the dataset
- Evaluate when serverless SQL (Athena) is the right tool versus when you need a persistent compute cluster (EMR)

Tasks

1. Configure Athena's query result location. Navigate to Athena in the console. If this is your first time opening Athena, you will see a banner asking you to set up a query result location. Choose Settings (gear icon in the top right), then Manage:

```
Query result location: s3://dsba6190-yourname-lab2/athena-results/  
Expected bucket owner: (leave blank)
```

Choose Save. This tells Athena where to store query output files.

2. Create a Glue database to organize your tables. In the Athena query editor, run:

```
CREATE DATABASE IF NOT EXISTS lab2_analytics;
```

3. Create an external table for the hourly aggregation. This DDL maps the Parquet files from Part 3 to a SQL-queryable table. Adjust the column names and types to match your Spark output schema:

```
CREATE EXTERNAL TABLE lab2_analytics.hourly_trips (  
  pickup_zone    STRING,  
  hour_of_day    INT,  
  trip_count     BIGINT,  
  avg_fare       DOUBLE,  
  avg_duration_min DOUBLE,  
  p90_fare       DOUBLE  
)  
STORED AS PARQUET  
LOCATION 's3://dsba6190-yourname-lab2/aggregated/hourly/'  
TBLPROPERTIES ('classification'='parquet');
```

4. Create an external table for the daily summary. Since this table is partitioned by date, include the **PARTITIONED BY** clause and then load partition metadata:

```
CREATE EXTERNAL TABLE lab2_analytics.daily_summary (  
  pickup_zone      STRING,  
  total_revenue    DOUBLE,  
  trip_volume      BIGINT  
)  
PARTITIONED BY (date STRING)  
STORED AS PARQUET  
LOCATION 's3://dsba6190-yourname-lab2/aggregated/daily_summary/'  
TBLPROPERTIES ('classification'='parquet');  
  
-- Load all partition metadata  
MSCK REPAIR TABLE lab2_analytics.daily_summary;
```

After running **MSCK REPAIR TABLE**, Athena will scan the S3 prefixes and register each `date=YYYY-MM-DD/` folder as a partition. You can verify by running **SHOW PARTITIONS lab2_analytics.daily_summary**;

5. Run three analytical queries. For each query, note the "Data scanned" and "Run time" metrics shown at the bottom of the results panel in Athena:

Query A — Top zones by volume:

```
SELECT pickup_zone, SUM(trip_count) AS total_trips  
FROM lab2_analytics.hourly_trips  
GROUP BY pickup_zone  
ORDER BY total_trips DESC  
LIMIT 10;
```

Query B — Average fare by hour (peak pricing analysis):

```
SELECT hour_of_day, ROUND(AVG(avg_fare), 2) AS mean_fare  
FROM lab2_analytics.hourly_trips  
GROUP BY hour_of_day  
ORDER BY hour_of_day;
```

Query C — Date-filtered query demonstrating partition pruning:

```
-- Run this first WITHOUT a date filter and note Data Scanned:  
SELECT pickup_zone, SUM(total_revenue) AS revenue  
FROM lab2_analytics.daily_summary  
GROUP BY pickup_zone ORDER BY revenue DESC LIMIT 5;  
  
-- Then run WITH a date filter and compare Data Scanned:  
SELECT pickup_zone, SUM(total_revenue) AS revenue  
FROM lab2_analytics.daily_summary  
WHERE date = '2023-01-15'  
GROUP BY pickup_zone ORDER BY revenue DESC LIMIT 5;
```

The filtered query should scan significantly less data because Athena only reads the Parquet files in the matching partition folder. This is the practical benefit of the partitioning you set up in the Spark job.

6. Write an analysis (8–12 sentences) addressing three things: (a) what the query results reveal about trip patterns in the dataset (which zones are busiest, when fares peak, how revenue distributes across zones); (b) how Parquet columnar storage and partitioning affected query performance based on the data-scanned metrics you recorded; (c) when

a data team should choose Athena for ad-hoc exploration versus standing up an EMR cluster for heavier processing, with specific examples from this lab.

Deliverables

- DDL statements for both external tables
- Screenshots of each analytical query with results, showing the Data Scanned and Run Time metrics
- Written analysis of trip patterns, partition pruning impact, and Athena vs. EMR tradeoffs

Final Cleanup

After completing Part 4, verify that all compute resources have been cleaned up. S3 data, Glue databases, and Athena tables can remain—they incur no ongoing compute cost.

- EKS cluster deleted (Part 2, Step 8)
- EMR cluster terminated (Part 3, Step 5)
- Docker-build EC2 instance stopped or terminated (Part 1)
- No SageMaker notebooks or endpoints running
- S3 data, ECR images, Glue databases, and Athena workgroups can remain (no compute cost)

Grading Rubric (75 points)

Component	Points	Weight
Part 1: Docker Data Cleaning Pipeline	15	20%
Part 2: EKS Parallel Processing	15	20%
Part 3: Spark Analytical Transformations	25	33%
Part 4: Athena Query Layer & Analysis	20	27%
Total	75	100%